Gower's distance combined with a context aware sequence distance metric

Suraj Tailor

August 2020

1 Introduction

This paper outlines a technique to cluster patient data consisting of the data types: categorical, binary, numeric and sequence. It was required that the distance metric for sequences described in the paper Context Aware Trace Clustering[1] was was implemented alongside or within a method to cluster mixed data. In the present paper, firstly, a comparison of different clustering techniques and distance metrics is outlined. Then, a justification for choosing the Gower's distance as the form of distance metric to cluster mixed attribute data is provided. Thereafter an explanation of how the Gower's Distance can be adapted to include sequence data is detailed. Then an explanation of the code used to implement this method within R is provided, including an overview of all of the functions involved in the code. Pen-ultimately, a suggestion on how to optimise the adapted Gower's distance function is given. Finally, this paper concludes by giving suggestions on further work to be carried out.

2 Techniques in clustering

In general, the clustering of sequential patient activity data along with their patient attributes largely presents a challenge due to the different distance metrics required to compare different types of data. Therefore a large overview of different clustering techniques is required in order to tackle these problems. Despite the fact the distance metric for sequences is already decided, it will still be analysed amongst other techniques.

2.1 K-means clustering

K-means clustering works on the basis of initially mapping an n-dimensional sample space onto a n-dimensional plane [2]. Thereafter, k points or centroids are selected within the plane, where $k \in \mathbb{N}$. These centroids do not have to be in the data set. Data points are assigned a cluster depending on how close they are to a centroid; this is determined by a distance metric of choice, normally euclidean distance. Notably k-means requires the user to select a specific number of centroids before clustering. This may require the user to have insight into the data or use a silhouette plot to find an optimum number of clusters. K-means is typically used on numerical data, but can be modified to be used on mixtures of data, which will be explained later. An adaptation of k-means is k-medoids where centroids are always data points in clustering.

2.2 Hierarchical clustering

Hierarchical clustering is similar to k-means in that it uses a user defined distance metric. It initially assumes all data points are its own unique cluster, then, by setting distances which incrementally get larger, it determines which clusters are shorter than that distance to one another [2]. Iteratively, the algorithm does this at increasing distances and logs the distances at which data points cluster together until all the data points are clustered. The output can be a dendrogram which makes it easier for the user to to make decisions on the optimum number of clusters to partition the data by eye. The approach just described is called the agglomerative approach or the bottom up approach. In a converse fashion, divisive hierarchical clustering, or the top down approach, can achieve the same result by starting off with all points classified as a single cluster and by increasing the number of clusters by division.

2.3 Sequential data clustering review

Clustering of sequence data is largely approached in two ways, similarity based and model based methods [1]. Similarity based methods utilise a distance function to see how closely related two sequences are. Model based methods work by making an assumption about the distribution of the

given data and iteratively updating the assumed parameters distribution using algorithms based on finding the maximum likelihood estimator. Note that model based methods can also be combined with similarity based methods.

2.3.1 Model based methods

Gaussian mixture models
Gaussian mixture models are one of the most popular model based clustering approaches available [3]. It assumes normally distributed sub-populations inside a population and iteratively attempts to find the maximum likelihood estimators of the distribution parameters using the EM algorithm until convergence is achieved. The number of distributions corresponds to the number of clusters desired.

Self organizing maps
Self Organizing Maps (SOM) are a type of simple neural network which maps high dimensional data to a lower dimensional map, often 2-D [4]. It utilises unsupervised learning to initially train a model using training data. Once trained, one can feed new data and it will be classified to a particular cluster. SOM's are useful in that they preserve the topological relationships in data. In other words, input data that are close in high dimensional space are also mapped to nearby nodes in the 2D map. The explanation of how SOM's work will not be provided in full due to its complexity, however it should be noted that probabilistic models can be implemented within SOM's to cluster sequence data which are of different length. This technique would consider the probability of an event occurring after another given event. These probabilities are determined by the input data. An example of this application is in [5].

2.3.2 Similarity based methods

This section describes different distance metrics for comparing two sequences. Once a distance matrix is found a host of methods could be used to cluster using the distance matrix, including k-means and hierarchical clustering.

Weighted Longest Common Subsequences (WLCS)
The WLCS method utilises ordered pairs consisting of time and sequence values for each event within a sequence [6]. The method then seeks to find a similarity measure between any two sequences in the data utilising information about the Longest Common Subsequence (LCS) between the two sequences. For example, the LCS between two sequences (A,B,C,B,D,A,B) and (B,D,C,A,B,A) would be (B,C,B,A). For any two sequences, the similarity measure is a function of two things. 1) The time spent on each event for the LCS for each sequence. 2) Time spent overall for each sequence. There are then two components that make up this similarity measure which are then multiplied to make a total similarity between two sequences. Suppose there are two sequences α and β where the cardinality of both sequences is greater than 0.

- **Similarity component:** the minimum time of an event from the LCS from either of the two sequences, divided by the maximum time of an event from the LCS from either of the two sequences. The average similarity between the sequences over the entire overlap region is calculated by dividing by the cardinality of the LCS. The similarity component formula is:

$$S' = \frac{1}{L} \sum_{i=1}^{L} \frac{min(t_{\alpha(i)}, t_{\beta(i)})}{max(t_{\alpha(i)}, t_{\beta(i)})} \tag{1}$$

  where $t_{\alpha(i)}$ is the time duration for a particular event of index i of the LCS for the α sequence, and $t_{\beta(i)}$ is the time duration for a particular event of index i of the LCS for the β sequence, and L is the cardinality of the LCS or the number of events in the LCS.

- **Importance component:** the importance the LCS has within each sequence relative to the time. This is a geometric mean described by the formula:

$$S'' = \left(\frac{T^{\alpha}_{LCS}}{T^{\alpha}} \times \frac{T^{\beta}_{LCS}}{T^{\beta}} \right) \tag{2}$$

  where T^{α}_{LCS} is total time spent in the LCS for the α sequence and T^{β}_{LCS} is the total time spent in the LCS for the β sequence, and T^{α} represents the total time in the α sequence, and T^{β} represents the total time in the β sequence.

The similarity between two sequences α and β can be given as:

$$S_{\alpha\beta} = S' \times S'' \tag{3}$$

The similarity of every pair of sequences can be calculated and stored into a matrix. This similarity matrix can be converted into a dissimilarity matrix using the readers preferred method of normalisation.

The advantage of this method is that it uses a similarity measure that looks at how common two sequences are based on multiple factors, namely time and their longest common sequence. The limitation of this approach is that since it is time dependent, depending on the nature of events within patient health, some events may have a great amount of variance in time which would effect the results. Another drawback is that sequences that are functionally the same may have a reduced similarity even though they should be very high in similarity, this is consequence in not considering the order of events when clustering.

Levenshtein distance
A fundamental measure of dis-similarity between two sequences is the Levenshtein distance. It determines the minimum number of edit operations needed to transform one sequence into the other. The transformation operations include an insertion, substitution or deletion of an event. Generally, a fixed cost is given to these operations and dynamic programming determines the overall minimum cost of transforming one sequence to another. The algorithm for determining Levenshtein distance consists of performing n x k sub problems where n and k are lengths of two sequences [7]. The Levenshtein distance between two sequences a, b (of length $|a|$ and $|b|$ respectively) is given by $lev_{a,b}(|a|, |b|)$ where

$$lev_{a,b}(i,j) = \begin{cases} \max(i,j) & if\ \min(i,j) = 0 \\ \\ lev_{a,b}(i-1,j)+1 & \\ lev_{a,b}(i,j-1)+1 & otherwise \\ lev_{a,b}(i-1,j-1)+1_{(a_i \neq b_j)} & \end{cases} \tag{4}$$

where $1_(a_i \neq b_j)$ is the indicator function equal to 0 when $a_i = b_j$ and equal to 1 otherwise. $lev_{a,b}(i,j)$ is the distance between the first i characters of a and the first j characters of b.

One major limitation of this method is that the Levenshtein distance does not consider the functional validity of any edit operation[1]. In other words, it does not consider if two event are similar to one another. It also means the order of events between sections of sequences are irrelevant.

2.4 Clustering categorical and numerical data

2.4.1 K-modes algorithm

The k-modes algorithm uses a simple matching dissimilarity measure to deal with categorical data. For any X,Y which are categorical data of the same dimension say n, the dissimilarity measure is defined by the number of mismatches of the corresponding attribute categories of two objects. It places a 0 if two objects match and 1 if they mismatch in to n dimension vector, the dissimilarity is then a count of this vector in $\mathbb{Z}^+$. Similar to k-means, k-modes has centroids, however these are defined as modes and is a vector of the same dimension as the number of categorical data attributes. The k-modes algorithm seeks to find the minimum distance between any set of k modes and all the objects assigned to one of these modes. The algorithm will try to find an optimised solution where the total distance between all cluster centres and data points are minimised.

2.4.2 K-prototypes algorithm

K-prototypes combines k-medoids and k-modes algorithms. It works firstly by combining the dissimilarity metrics in both of these algorithms to make a new metric as shown in [8]. Then, similar to k-modes, the algorithm seeks to find an optimised solution where the total distance between all cluster centres and objects assigned to each centre is minimised. For more information on this method, read [8]. If a R function is required to perform k-prototypes, refer to [9].

2.4.3 Gower's distance explanation

Gower's distance produces a similarity matrix for data types with a mixture of binary, categorical, numeric and ordered variable types. Instead of a similarity measure described in the original Gower's distance [10], a dissimilarity measure as shown in [11] which gives distances between [0,1] is described below. For each data type, a unique distance metric is used. The governing equation for each of the values in the distance matrix is:

$$D(i,j) = \frac{\sum_k^p \delta_{ijk} d_{ijk} w_k}{\sum_k^p \delta_{ijk} w_k} \tag{5}$$

where $D(i,j)$ corresponds to the final dissimilarity between the ith and jth patient. w_k is the weight assigned to a particular data attribute k; these weights determine how much importance is given to a attribute in the distance. d_{ijk} is the distance between the ith and jth patient considering the kth variable. δ_{ijk} is a function which is 1 when both units contain the data and thus can be compared, is set to 0 when at least one of the units does not contain that data. p refers to the number of data attributes.

The dissimilarity function, d_{ijk} changes according to the data type:

- Logical variables or Binary: Logical variables are considered asymmetric binary variables. For this case $d_{ijk} = 0$ if $x_{ik} = x_{jk} = TRUE$, 1 otherwise.
- Categorical nominal variables: $d_{ijk} = 0$ if $x_{ik} = x_{jk}$, 1 otherwise.
- Numeric variables: $d_{ijk} = \frac{|x_{ik} - x_{jk}|}{R_k}$ where R_k is the range of the kth variable.
- Ordered variables: $z_{ik} = \frac{(r_{ik}-1)}{max(r_{ik})-1}$ where r_{ik} are ordered values and z_{ik} are values that are treated as values as a numeric variable.

There is no distance function considered for sequence data types in the Gower's metric. Therefore, in order to generate a Gower's distance matrix for mixed data including sequence data, it is required that a sequence distance metric is first defined and then added to existing Gower's distance functions.

3 Context aware distance (chosen sequence distance)

The method to find distance between sequences is the same as shown in the paper Context Aware Trace Clustering [1], this method shall be called the **context aware distance** from here forth. The context aware distance is an adaptation of the Levenshtein distance shown in the previous section. This distance works by finding the similarity between any two sequences for all the events. It then converts these similarities into distances. In this metric, instead of assigning a cost for substituting, deleting or inserting events into a sequence, it calculates a score in performing these operations. The algorithm encourages high scoring operations through out it. The scores are based on the probability of events being inserted, deleted or substituted given a certain context. These scores are determined from analysing the entirety of the data and counting the occurrence of every event in specific contexts.

Namely, there are three $\mathbb{A} \times \mathbb{A}$ matrices which these scores come from, where $\mathbb{A}$ refers to the set of events. i) substitution matrix which corresponds to the similarity of substituting one event with another. ii) insertRightGivenLeft matrix which corresponds to the likelilhood of inserting an event to the right of a given event. iii) insertLeftGivenRight matrix which corresponds to the likelihood of inserting an event to the left of a given event. Both insertion matrices are not required to calculate similarities, although both can be used. In this paper only the insertRightGivenLeft will only be used along with the substitution matrix for calculating similarities between t sequences. Also it is worth noting that the Levenshtein distance uses the operations of insertion, deletion, and substitution. Since deletion and insertion are complementary, only the operations substitution and insertions are considered for the context aware distance.

The similarity between two sequences S^m and T^n, with lengths $M \geq 1$ and $M \geq 1$ and where the superscript refers to the sub sequence of S and T, can be defined as:

$$Sim(S^m, T^n) = max \left\{ \begin{array}{l} s(S(m), T(n)) + Sim(S^{m-1}, T^{n-1}) \\ i(S(m), -) + Sim(S^{m-1}, T^n) \\ i(-, T(n)) + Sim(S^m, T^{n-1}) \end{array} \right\} \tag{6}$$

where $s : (\mathcal{A} \times \mathcal{A}) \rightarrow \mathcal{R}$ defines substitutions scores where $\mathcal{A}$ is the set of events and $\mathcal{R}$ is the set of real numbers. $S(m)$ and $T(n)$ refer to the index at m and n for each sequence and is a single event. $s(S(m), T(n))$ defines the scores for substituting $S(m)$ with $T(n)$. i defines the insertion and deletion insertRightGivenLeft scores. $i(S(m), -)$ defines the score for deleting $S(m)$ while $i(-, T(n))$ defines the score for inserting $T(n)$. For more information on this similarity, refer to [1]

The individual values of the matrix used to calculate the similarity $Sim(S, T)$ can be defined as $F(i,j)$ where i, and j are the indexes of this matrix where $i \geq 0$ and $j \geq 0$. The dimensions of

this matrix would be $(m+1) \times (n+1)$. Then, each value of $F(i,j)$ is calculated by the following equation:

$$F(i,j) = max \left\{ \begin{array}{l} S(S(i), T(j)) + F(i-1, j-1) \\ I(S(i), S(j-1)) + F(i-1, j) \\ I(T(j), T(j-1)) + F(i, j-1) \end{array} \right\} \tag{7}$$

Where S refers to the substitution matrix, I is the insertRightGivenLeft matrix. There are also important initial conditions that need defining. Firstly, it should be noted that any index with a 0 in it, refers to a sub sequence of 0 events for one of S or T. The initial conditions are defined below:

$$F(0,0) = 0 \tag{8}$$

$$F(1,0) = 0 \; and \; F(0,1) = 0 \tag{9}$$

$$for \, all \, i > 1 \; F(i,0) = F(i-1,0) + I(S(i), S(i-1)) \tag{10}$$

$$for \, all \, j > 1 \; F(0,j) = F(0,j-1) + I(T(j), T(j-1)) \tag{11}$$

The insertLeftGivenRight matrix could similarly be applied but by performing the Equation 7 from the right to left of the sequence instead, and therefore the matrix for calculating the alignment will start at the bottom right instead of the top left. For more details about how these functions work, refer to [12].

Once a similarity matrix is obtained by calculating all the pairwise combinations of sequence similarities, a distance matrix can be obtained by using the equation below:

$$d(S,T) = \frac{|S| + |T|}{Sim(S,T)} \tag{12}$$

where $|S|$ and $|T|$ are the cardinality of each sequence. To ensure this distance function outputs positive values, one can shift the substitution scores and indel scores by adding the absolute minimum value to each matrix, thereby shifting the values of the substitution and indel scores such that they are all positive. This shifting is as suggested in [13]. Alternatively, one can also shift the final distance matrix by adding the absolute minimum value to all values in the matrix.

4 Gower's distance adaptation (chosen method)

The chosen method for clustering the mixed data was an adaptation of the Gower's distance described in Section 2.4.3. The following steps were carried out:

1. The Gower's distance metric is adapted to include the context aware distance as described in Section 3.
2. The adapted Gower's distance metric is applied to patient data and a distance matrix is obtained.
3. Hierarchical clustering is applied to the distance metric to find all the combinations of clusters for the data.
4. A silhouette plot is used to obtain the optimum number of clusters for the data.

Clustering is then complete.

5 Other mixed data clustering approaches

An adaptation of the k-prototypes method could be considered by first adding context aware distance proposed in Section 3 and then applying the optimisation algorithm in three parts versus the original two as proposed in [8]. This method was rejected since the dendrogram resulting from hierarchical clustering is used in the algorithm to align the sequences and display this alignment in the visualisation software.

All Model based methods are disregarded because an understanding of the domain is required to estimate the type of distribution that the data best represents. Furthermore, there would be complexity in incorporating the chosen context aware distance into these methods.

6 Gower's distance adaptation

There is no existing Gower's distance metric in R libraries that incorporates the context aware distance as shown in Section 3. Therefore it is necessary to build or adapt existing Gower's distance metric code such that it can do this. There are already robust programs which can apply the Gower's distance effectively for all other types of data such as daisy package. Below details the steps required to add the context aware distance values to an application of the daisy function in R such that it conforms to the scaling of distance inside Gower's distance. Namely, the distance metrics must have values between [0,1].

Step 1: Apply context aware distance as described in Section 3 to all sequence data to obtain context aware distance matrix containing all distance between all sequences. Then shift context aware distances such that they are all positive as proposed in Section 3.

Step 2: Apply the scaling to the context aware distance matrix such that they bound between [0,1].

$$J = \frac{M - Min(M)}{Max(M) - Min(M)} \tag{13}$$

Where J is the scaled context aware distance matrix and M is the original context aware distance matrix. This is a generic scaling between [0,1]. Through this scaling, none of the information should be lost. The only effect in reducing or increasing values would be in the significance when the final Gower's distance matrix including all the data is clustered. This effect can be adjusted through applying weights.

Step 3: Rearrange the Gower's distance with all data attributes such that it can be easily applied to the daisy function. First the distance between two patients is described by the equation below:

$$D(i,j) = \frac{\sum_k^{p+1} \delta_{ijk} d_{ijk} w_k}{\sum_k^{p+1} \delta_{ijk} w_k} \tag{14}$$

This can be rearranged to form:

$$D(i,j) = \frac{\sum_k^{p} \delta_{ijk} d_{ijk} w_k + \delta_{ij(p+1)} d_{ij(p+1)} w_{p+1}}{\sum_k^{p+1} \delta_{ijk} w_k} \tag{15}$$

where the $\sum_k^p \delta_{ijk} d_{ijk} w_k$ term is the numerator of the ith and jth value of the Gower's distance matrix without the sequence data. The $\delta_{ij(p+1)} d_{ij(p+1)} w_{p+1}$ term relates to the indicator function δ, context aware distance function d and weight w respectively for the ith and jth patients. The denominator $\sum_k^{p+1} \delta_{ijk} w_k$ is the summation of of all the indicator functions for all the ith and jth patient multiplied by each of their respective weights for each data type.

Since there are no existing functions in R which include this distance metric, a rearrangement of this formula is provided so that the existing daisy function in R can be utilised. The final equation is a rearrangement of Equation 14 such that the Gower's distance is an isolated term. This way the daisy function can then be applied to all the data not including the sequence data. The other components around this term would then need to be calculated. This final equation is built up with the following equations A, B and C described below:

$$A = \frac{\sum_k^{p} \delta_{ijk} w_k}{\sum_k^{p+1} \delta_{ijk} w_k} \tag{16}$$

$$B = \frac{\delta_{ij(p+1)} d_{ij(p+1)} w_{(p+1)}}{\sum_k^{p} \delta_{ijk} w_k} \tag{17}$$

$$C = \frac{\sum_k^{p} \delta_{ijk} d_{ijk} w_k}{\sum_k^{p} \delta_{ijk} w_k} \tag{18}$$

where C is the Gower's distance for p data attributes. The numerator of B contains the distance metric, weight and indicator function for the sequence data. The denominator of B contains the summations of weights for all data apart from the sequence data. The A term has the summation of all weights apart from the sequence data on the numerator and the denominator contains a summation of all the weights of the data including the sequence data. The final equation is then:

$$D(i,j) = A * (C + B) \tag{19}$$

7 Explanation of mainScript.R code

The implementation of the method described in Section 4 was written in R. The files can be found on GitHub here. Once the files have been fetched, the mainScript.R is the script that defines all the global variables, sources all the functions required from scripts folder and ultimately, runs the clustering for patient data which has sequence data. The patient data should be an input in the form of a .csv.

```
18 this.dir <- dirname(getActiveDocumentContext()$path)
19 setwd(this.dir)
20
21 #Initialize global variables
22 FN_INPUT<-"/data/testdata.csv"
23 FN_OUTPUT<-"/data/outputClustering.csv"
24 FN_SUFFIX<-"CHCData"
25 MAX_NUMBER_CLUSTERS<<-50 #compute the average silhoue
26 PLOT_SILHOUETTE<<-TRUE
27 PLOT_DENDROGRAM<<-TRUE
28 SAVE_PLOT_SILHOUETTE<<-FALSE
29 REMOVE_SEQ_FOR_CLUSTERING<<-TRUE #take only the most
30 N<<-30 #number of sequences as input to clustering. c
31 CARDINALITY_EVENTS <- 12 #number of unique elements i
32 SHIFT_SCORES <- TRUE
```

Figure 1: Assigning global variables.

The location of the main script is set as the working directory. FN_INPUT is the directory of the csv file containing all of the patient data. FN_OUTPUT is the output file for the clustering allocation of each patient as a .csv file. MAX_NUMBER_CLUSTERS is the max number of clusters computed when creating silhouette plots. N is the number of sequences to take for clustering. The CARDINALITY_EVENTS is set to the number of events within the sequences plus one. This addition of one is to account for a blank event which is appended to sequences of length ≤ 1 in the fillSeq function. The rest of the variables are logical and are used to turn on functions within the script.

```
42 #Data cleaning
43 uniqueSequences<-read.csv(sprintf("%s%s",getwd(),FN_INPUT))
44 filteredUniqueSequences<-reduceDataSize(uniqueSequences, REM
45 splitSeq <- fillSeq(filteredUniqueSequences, CARDINALITY_EVE
46 gramDataFrame <- find3Grams(splitSeq, filteredUniqueSequence
47 uniqueThreeGrams<-findUnique3Grams(gramDataFrame) #tested
48 totalFreq3Grams <- calcTotalFreq3Grams(gramDataFrame,uniqueT
```

Figure 2: Data cleaning functions.

The reduceDataSize function reduces the data to the top N clusters with the highest frequency within the patient data. The fillSeq function adds a blank event to sequences with only one or two events. This is important as the algorithm for finding the 3-grams requires the sequences to have at least three events. This blank event is equal to the value of CARDINALITY_EVENTS. The find3Grams function finds all the 3-grams within each sequence and determines their frequency within that sequence. The findUniqueSequences function finds all the unique 3-grams across all the sequences. The calcTotalFreq3Grams function finds the frequency of each 3-gram across all the sequences.

```
50 #Substitution matrix
51 coCount<-calcCoCountMatrix(totalFreq3Grams, CARDINALITY_EVE
52 normalizedCoCount<-calcNormalizedCoCount(coCount) #tested
53 coCountProbVector<-calcCoCountProbVector(normalizedCoCount)
54 coCountExpMatrix<-calcCoCountExpMatrix(coCountProbVector) #
55 subsMatrix<-calcSubsMatrix(normalizedCoCount, coCountExpMat
```

Figure 3: Substitution matrix functions.

All the functions for calculating the substitution matrix relate to the steps found in Algorithm 1 in [1]. The calcCoCountMatrix determines the co-count for any two events, step 6. The calcNormalizedCoCount function determines sum of all co-counts across all events. The calcCoCountProbVector function determines the probability of a single event occurring, step 9. The calcCoCountExpMatrix function determines the expected occurrence of any two events, step 10. The

calcSubsMatrix function determines the matrix of scores using the log-odds ratio, step 11. The output is a symmetric matrix. Note that the diagonals do not equal zero, this is due to the nature of the algorithm and is not a mistake.

```
57  #CountRGL and LGR
58  countRGL <- getCountRGL(CARDINALITY_EVENTS, totalFreq3Grams
59  countLGR <- getCountLGR(CARDINALITY_EVENTS, totalFreq3Grams
```

Figure 4: Count RGL and LGR functions.

The functions in Figure 4 relate to the step 5 found in Algorithm 2 in [1]. The getCountRGL function counts the occurrences of an event given an particular event is to the left of it. Similarly, the getCountLGR function counts the occurrences of an event given a particular event is to the right of it. Both of these functions output a matrix with dimensions CARDINALITY_EVENTS × CARDINALITY_EVENTS.

```
61  #InsRGLMatrix
62  RGLNormVector <- calcInsNormVector(countRGL) #tested
63  RGLProbVector <- calcInsProbVector(RGLNormVector) #not test
64  normRGLMatrix <- calcNormInsMatrix(RGLNormVector, countRGL)
65  insRGLMatrix <- calcInsMatrix2(normRGLMatrix, RGLProbVector
66
```

Figure 5: InsertRightGivenLeft functions.

The functions in Figure 5 relate to steps 6 to 9 in Algorithm 2 in [1]. The calcInsNormVector function calculates the sum of each row in the countRGL matrix, step 6. The calcInsProbVector determines the probability of an event a occurring, defined as the norm of an event divided the sum of the norm of all events. The output is a matrix which is read by taking the row as the *Right* event and the *given Left* event are the columns. Therefore, (1,2) would be the event 1 occurring given that to the left of it is event 2.

```
67  #InsLGRMatrix
68  LGRNormVector <- calcInsNormVector(countRGL)#repeat
69  LGRProbVector <- calcInsProbVector(LGRNormVector)#repeat
70  normLGRMatrix <- calcNormInsMatrix(LGRNormVector, countLGR
71  insLGRMatrix <- calcInsMatrix2(normLGRMatrix, LGRProbVecto
72
```

Figure 6: InsertLeftGivenRight functions.

The functions in Figure 6 are the same functions in Figure 5, applied instead to countLGR matrix.

```
73  #Shift scores to ensure distances are >0
74  if(SHIFT_SCORES == TRUE ){
75  shiftScoresOutputs <- shiftScores(insRGLMatrix,insLGRMatrix,
76  subsMatrix <- shiftScoresOutputs[[1]]
77  insRGLMatrix <- shiftScoresOutputs[[2]]
78  insLGRMatrix <- shiftScoresOutputs[[3]]
79  }
```

Figure 7: Shifting scores function.

The shiftScore function finds the absolute value of the minimum of a matrix. It then adds this value to each matrix. This method ensure that the final distances are all positive. This method is as described in [13], in Section 4 of the reference paper.

```
81  #SeqDistance matrix function
82  seqDistancesRGL <- getGenericEditDistanceRGL(splitSeq, insRGLMatri
83  seqDistancesLGR <- getGenericEditDistanceLGR(splitSeq, insLGRMatri
```

Figure 8: Sequence distance functions.

The getGenericEditDistanceRGL calculates the context aware distance between two sequences according to [12], in Section 3 of the referenced paper. The insertions and deletions are based only off of the insertionRightGivenLeft matrix. Similarly, the insertions and deletions in getGenericEditDistanceLGR are determined from the insertionRightGivenLeft matrix. Both insertion matrices can be incorporated, however, this would increase the computational complexity. Therefore only

one type of insertion is chosen for usage in the getGenericEditDistance functions. The getGenericEditDistanceLGR adopts a bottom up approach and calculates the similarity starting from the bottom right of the alignment matrix.

```
89  #Final distance function
90  finalDistanceMatrixRGL <- getGowerDistance(seqDistancesRGL,
91  finalDistanceMatrixLGR <- getGowerDistance(seqDistancesLGR,
92  #print('Created finalDistanceMatrix.')
```

Figure 9: Gower's distance functions

The getGowerdistance function calculates the Gower's distance for all the data except the sequence data. Then it calculates the Gower's distance of all the data including the sequence data. This is calculated according to the equation 19. Presently, sample data for all different types of attribute data, not including sequence data, is stored within this function.

```
94  ######################################################
95  #Obtain max number of clusters.
96  maxNumberClusters <- getMaxNumClusters(length(splitSeq)) #no
97
98  #Clustering processes.
99  clusteringInfo <- clusterUniqueSequences(finalDistanceMatrixI
100 bestK <- clusteringInfo[[1]]
101 hc <- clusteringInfo[[2]]
102
103 #Cut tree using best number of clusters
104 clusteredUniqueSeq <- treeCut(hc, bestK, uniqueSequences, FN_
105 print(clusteredUniqueSeq)
106
107 #Plot dendogram
108 plotDend(PLOT_DENDOGRAM,hc,bestK) #not tested
```

Figure 10: Clustering functions.

The getMaxNumberClusters function limits maximum number of clusters to be one less than the number of unique sequences. The clusterUniqueSequences function first performs hierarchical clustering on the distance matrix, then silhouette clusters are calculated iteratively to find the optimum number of clusters and the results are then plotted on a silhouette plot. Then, the optimum number of clusters and the hierarchical clustering results is outputted. The treeCut function then takes the optimum number of clusters and the hierarchical information to assign each patient to a cluster. The output is a dataframe with each unique patient along with the assigned cluster. Finally, the plotDend function plots a dendrogram of the hierarchical information.

```
86  #Alternate sequence distance function
87  seqDistanceRGLAlternate <- getGenericEditDistanceRGLAlternate(
```

Figure 11: Clustering functions.

The getGenericEditDistanceRGLAlternate is an implementation of the getGenericEditDistance where the insertRGL operations are performed across different sequences. For example insRGLMatrix(firstSequenceIndex, secondSequenceIndex). The first row and column used to calculate the alignment is initialized by taking every value to be 0.

8 Explanation of certain functions

This section explains in more detail certain functions used on the mainScript.R.

```r
#Adds blank events to sequences of length less than three.
fillSeq <- function(filteredUniqueSequences, CARDINALITY_EVENTS){

  splitSeq<- as.character(filteredUniqueSequences[,"sequence"])

  for(i in 1:length(splitSeq)){

    # Attaches one blank event to the end.
    if(nchar(splitSeq[i]) == 3){
      splitSeq[i] <- paste0(splitSeq[i], c(sprintf('-%s',CARDINALITY_EVENTS)))
    }

    # Attaches two blank events to the end.
    else if(nchar(splitSeq[i]) == 1){
      splitSeq[i] <- paste0(splitSeq[i], c(sprintf('-%s-%s', CARDINALITY_EVENTS, CARDINALITY_EVENTS)))
    }
  }
  return(splitSeq)
}
```

Figure 12: fillSeq adds blank events to sequences of length less than 3.

(12) above is the fillSeq function which adds the event with the value CARDINALITY_EVENTS to sequences with less than three events. Three grams cannot be generated for sequences of less than three and returns an error if this is attempted inside the R function n-grams.

```r
43      #Determine [1,] values of edit matrix correct
44      for(l in 3:matrixRow){
45
46        #Iteratively update rows going down
47        editMatrix[l,1] <- insRGLMatrix[firstSeqVectorNum[l-1], firstSeqVectorNum[l-2]] + editMatrix[l-1,1]
48      }
49
50      #Determine [,1] values of editmatrix
51      for(m in 3:matrixCol){
52
53        #Iteratively update columns going across
54        editMatrix[1,m] <- insRGLMatrix[secondSeqVectorNum[m-1], secondSeqVectorNum[m-2]] + editMatrix[1, m-1]
55      }
56
57      #Determine [2,2] special case of editmatrix
58      subCost<- subsMatrix[firstSeqVectorNum[1],secondSeqVectorNum[1]] + editMatrix[1,1]
59      editMatrix[2,2] <- max(0, subCost)
60
61      #Determine second row of edit matrix
62      for (p in 3:matrixCol) {
63
64        subCost <- subsMatrix[firstSeqVectorNum[1], secondSeqVectorNum[p-1]] + editMatrix[1,p-1]
65        insertionRGLCost <-  insRGLMatrix[secondSeqVectorNum[p-1], secondSeqVectorNum[p-2]] + editMatrix[2,p-1]
66        deletionRGLCost <- editMatrix[1,p]
67
68        editMatrix[2,p] <- max(subCost, deletionRGLCost, insertionRGLCost)
69      }
70
71      #Determine second column of edit matrix
72      for (q in 3:matrixRow){
73
74        subCost <- subsMatrix[firstSeqVectorNum[q-1], secondSeqVectorNum[1]] + editMatrix[q-1,1]
75        insertionRGLCost <- editMatrix[q,1]
76        deletionRGLCost <- insRGLMatrix[firstSeqVectorNum[q-1], firstSeqVectorNum[q-2]] + editMatrix[q-1,2]
77
78        #assign the maxcost to the element within editmatrix
79        editMatrix[q,2] <- max(subCost, deletionRGLCost, insertionRGLCost)
80      }
81
82      #Rest of algorithm row by row starting [3,3]
83      for(n in 3:matrixRow){
84
85        for(o in 3:matrixCol){
86
87          subCost <- subsMatrix[firstSeqVectorNum[n-1],secondSeqVectorNum[o-1]] + editMatrix[n-1,o-1]
```

Figure 13: Base conditions used inside the getGenericEditDistanceRGL function.

Assuming the index's of matrices start from 1 in R and not 0, the similarity matrix assumes that the values with index (1,1), (1,2), (2,1) are all zero. These initial values are obtain from the paper [12]. Lines 45-48 initializes the first column of the similarity matrix by working out all of the indels for the first sequence, there is no application of the substitution matrix in this instance. Similarly lines 51-55 initializes the first column of the similarity matrix. Lines 58-59 calculates a special case of the similarity matrix at the location (2,2). Lines 61-80 calculates the second row and column of the similarity matrix. These are separately calculated because the $i(a,-)$ and $i(-,a)$ values as shown in Section 3, where a is an arbitrary event, are defined to be zero. Finally, lines 83 on wards calculates the rest of the matrix using the Equation 7 in Section 3. A analogous method is used in the getGenericEditDistanceLGR function, however the alignment matrix begins from the bottom right and it is assumed the sequences are being aligned starting from the right to left instead of left to right.

9 Optimising the Gower's distance weights

Currently the weights for the Gower's distance, w_k, all have a value of 1. These weights can be adjusted manually according to which data the user decides is more important in influencing the clustering. Alternatively, there are automated ways to determine optimal clusters sizes. The method shown below have not been coded.

Average Sillouhette Width (ASW) approach

The weights of the Gowers's distance can be optimized to obtain a higher Average Silhouette Width (ASW) value of the clustering results. This method is presented in [14]. ASW can be used to measure the quality of clustering results in which a high value leads to higher intra-cluster homogeneity and inter-cluster dissimilarity. Therefore, maximizing the ASW value improves the quality of the clusters which is the goal of optimisation. This ASW reflects both compactness and separation of clusters. The process is described below.

Firstly, the optimal number of clusters that has the highest ASW value is found. This is the same as finding the optimum number of clusters using a silhouette plot. There after, for an optimal number of cluster k, the ASW value for user defined weight is found. The default is for all weights to be 1. This can be described by Algorithm 1 below, where *distances* is the distance matrix after Gower's distance is applied to all the data.

Algorithm 1: Function A to find the ASW value of the clustering with the input weights

Input: weights
Output: ASW value

1. Calculate a Gower's distance matrix with the weights
   $Gower_dist \leftarrow daisy(distances,\ weights = weights)$
2. Partition the cases into k clusters
   $clusters \leftarrow pam(Gower_dist,\ k)$
3. Calculate the ASW value of the cases in the k clusters
   $ASWvalue \leftarrow clusters\$silinfo\$avg.width$
4. **return** -ASW value

Secondly, the default Gower's distance weights are optimised to obtain a higher ASW for the optimal number of clusters. The optim function in R is used in Algorithm 2 for this purpose. The optim function takes the initial parameters, and it tunes them to minimize the return of the target function A. Even though a higher value of function A is desired, optim works by finding the minimum, therefore the output ASW value from function A must be converted to a negative. Function B then works iteratively to return optimal weights. The user can define upper and lower bounds for the weights. This can be set to 1 and 3 for the lower and upper weights respectively. This means weights can be set to any value between these two values. Since the weights are relative, one can set the lower bound to 1 for comparing to the default weights and an upper bound of 3, in order to ensure one data type does not dominate.

Algorithm 2: Function B to optimize the default weights

Input: default weights, upper bounds(u), lower bound(l)
Output: optimized weights

1. Calculate the optimized weights through the optim function.
   $optimized_weights \leftarrow optim(par = weights, fn = functionA, lower = 1, upper = u, method =' L - BFGS - B')$
2. **return** optimized_weights

Once this optimization is complete, the calculations for the new Gower's distance and clustering would have already been performed within Algorithm 1. It is simply a case of extracting the relevant information.

One limitation of this approach is in setting an upper bound for the weights. It might be the case that the weights may increase beyond the upper limit. With a broader range of results, a higher ASW value can be obtained. The problem with this is in higher computational time when increasing the range between the lower and upper limits.

10 Further work

This method's effectiveness needs to be tested on data where the user has a good understanding of the domain. This way the user would be able to understand how the data should ideally be clustered. Furthermore a test comparing the effectiveness of different type of sequence metrics which are incorporated inside the Gower's distance could be conducted. Another test could be conducted where the sequence distance metric could be kept the same as in Section 3, but a different method to cluster the mixed data could be implemented.

There are certain additions and adaptations to the code that have not been implemented that may be useful. Firstly, one could add a method to automatically detect the number of unique events and determine the cardinality of unique events. This approach could potentially save computational time when calculating the substitution and insertion matrices. Secondly, the method to optimise the weights could be implemented as described in Section 8. Thirdly, there needs to be simple way to set the weights of each of the data attributes, at the moment the default is set to 1 with no way to change this. Finally, an option to shift the context aware distance matrix such that all the values are positive might be an alternative to shifting the substitution and insertion matrices.

These functions still need testing: getGenericEditDistanceRGLAlternate, getGenericEditDistanceLGR, getMaxNumClusters, clusterUniqueSequences, treeCut and plotDend.

These functions would benefit from further testing: getGowerDistance.

References

[1] J. C. B. R.P. and W. Aalst, "Context aware trace clustering: Towards improving process mining results," Apr. 2009. DOI: 10.1137/1.9781611972795.35.

[2] S. K. is a Data Science enthusiast, *Clustering: Types of clustering: Clustering applications*, Jul. 2020. [Online]. Available: https://www.analyticsvidhya.com/blog/2016/11/an-introduction-to-clustering-and-different-methods-of-clustering/.

[3] B. Boehmke and B. M. Greenwell, *Hands-on machine learning with R*. CRC Press, Taylor & Francis Group, 2020.

[4] J. A. Bullinaria, *Introduction to neural computation (level 4/m)*. [Online]. Available: https://www.cs.bham.ac.uk/~jxb/inc.html.

[5] J. Wei, Z. Shen, N. Sundaresan, and K.-L. Ma, "Visual cluster exploration of web clickstream data," *2012 IEEE Conference on Visual Analytics Science and Technology (VAST)*, 2012. DOI: 10.1109/vast.2012.6400494.

[6] A. Banerjee and J. Ghosh, "Clickstream clustering using weighted longest common subsequences," Aug. 2001.

[7] B. Aurélien, A. Habrard, and M. Sebban, *Metric learning*. Morgan & Claypool, 2015.

[8] J. Huang, "Extensions to the k-means algorithm for clustering large data sets with categorical values," *Data Mining and Knowledge Discovery*, vol. 2, pp. 283–304, 1998.

[9] G. Szepannek, "Clustmixtype: User-friendly clustering of mixed-type data in r," *The R Journal*, vol. 10, p. 200, Jan. 2019. DOI: 10.32614/RJ-2018-048.

[10] J. C. Gower, "A general coefficient of similarity and some of its properties," *Biometrics*, vol. 27, no. 4, p. 857, 1971. DOI: 10.2307/2528823.

[11] *Statmatch*. [Online]. Available: https://www.rdocumentation.org/packages/StatMatch/versions/1.3.0/topics/gower.dist.

[12] R. J. C. Bose and W. M. V. D. Aalst, "Process diagnostics using trace alignment: Opportunities, issues, and challenges," *Information Systems*, vol. 37, no. 2, pp. 117–141, 2012. DOI: 10.1016/j.is.2011.08.003.

[13] W. V. D. Aalst, "Process mining in the large," *Process Mining*, pp. 353–385, 2016. DOI: 10.1007/978-3-662-49851-4_12.

[14] A. Bektas and R. Schumann, "How to optimize gower distance weights for the k-medoids clustering algorithm to obtain mobility profiles of the swiss population," *2019 6th Swiss Conference on Data Science (SDS)*, 2019. DOI: 10.1109/sds.2019.000-8.